JOHANNES KEPLER
UNIVERSITY LINZ

Clock Domains, Metastability, and Resets



Simon Dorrer

KV VLSI Design, 336.051, WiSe 26/27

IICQC
INSTITUTE FOR
INTEGRATED CIRCUITS &
QUANTUM COMPUTING

JOHANNES KEPLER
UNIVERSITÄT LINZ
Altenberger Straße 69
4040 Linz, Österreich
jku.at

Today's Goals

After this lecture, you can

- find the **clock domain crossings** of a design
- explain **metastability** and compute an **MTBF**
- synchronize a **level** and a **pulse**
- cross a **bus** with Gray code, handshake, or **asynchronous FIFO**
- simulate crossings with **metastability injection**
- constrain crossings in **SDC**
- release a reset with a **reset synchronizer**



Chapter 8

Lecture notes: Chapter 8 of iic-jku.github.io/vlsi-design

Clock Domains



Clock Domains and Crossings

- **Clock domain**: the registers driven by one clock
- **Synchronous** clocks: one source, known phase, STA times the paths (Chapter 6)
- **Asynchronous** clocks: different sources or unknown phase, the phase drifts
  - processor, SPI master outside, USB 48 MHz, real-time clock 32.768 kHz, ADC
- Every asynchronous input pin is a crossing as well
- A **clock domain crossing** (CDC) starts in one domain and ends in another
  - the capturing flip-flop sooner or later sees its data change inside its setup and hold window

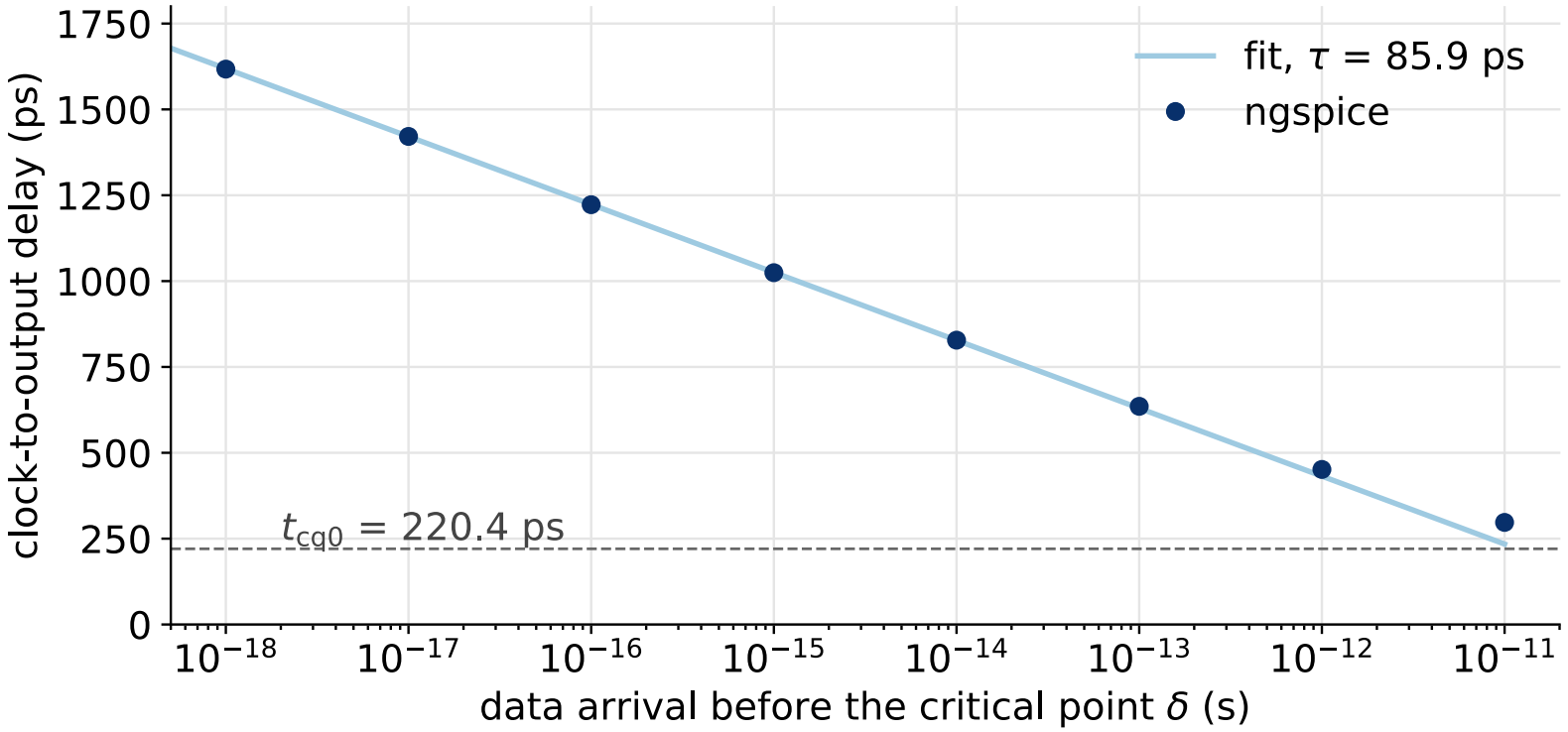
Metastability



A Flip-Flop Between 0 and 1

- Master latch: two **cross-coupled inverters**, two stable states, a **balance point**
- `D` changes just as the master closes: the loop starts near the balance point
- The difference grows as exp(t/τ), with the time constant τ of the loop
- The closer the data edge, the **longer** the flip-flop needs
  - output between 0 and 1, or a late edge, two flip-flops behind it may disagree
- Measured in **ngspice** on the transistor netlist of `dfrbpq_1`: `meta/dff_meta.py`
  - the PDK netlist is the schematic: plus an estimated 1.5 fF per internal node

Measured in ngspice



- $t_{cq} = t_{cq0} + \tau \ln(T_0 / 2\delta)$: **$\tau$ = 85.9 ps**, **T_0 = 23.5 ps**, t_{cq0} = 220.4 ps, typical corner

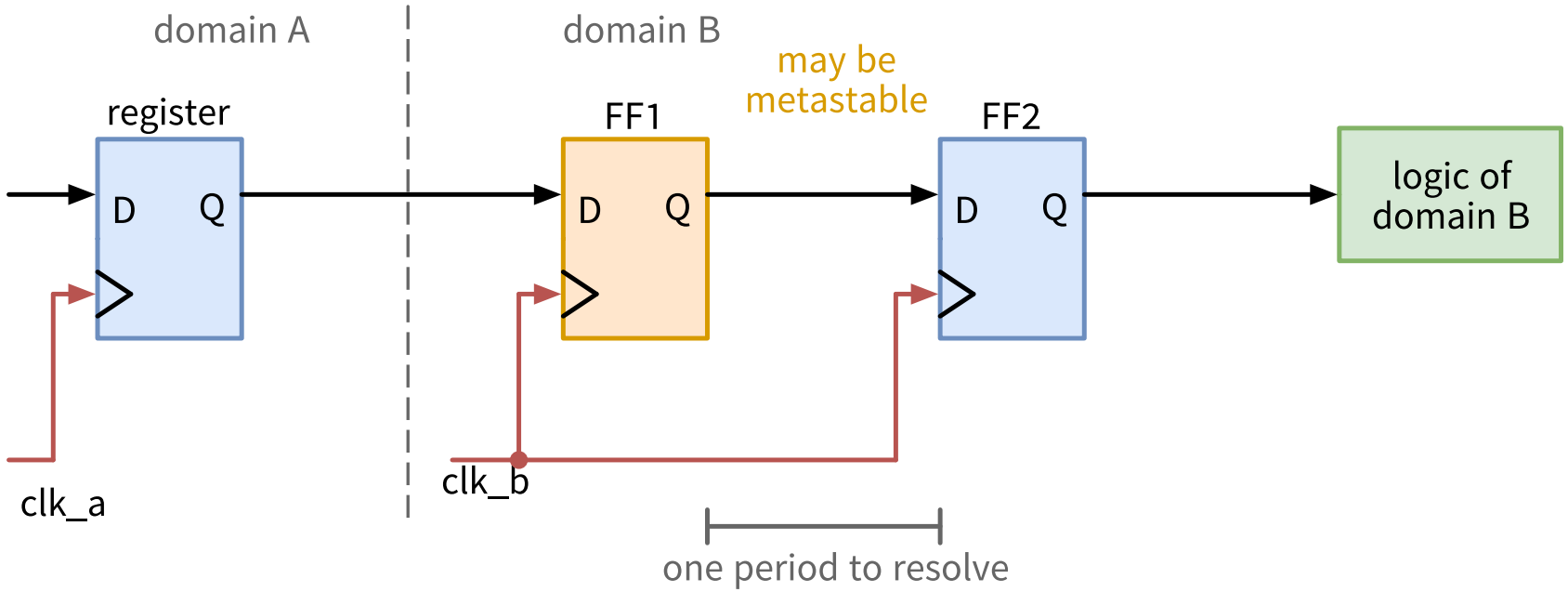
Mean Time Between Failures

- More than t_r extra if the data edge hits a window $T_0 \cdot \exp(-t_r/\tau)$
- **MTBF = $\exp(t_r/\tau) / (T_0\, f_{clk}\, f_{data})$**
- $t_r = T - t_{cq0} - t_{logic} - t_{su}$: what is left of the period
- T_0 and τ: properties of the flip-flop and its corner, **T_0 is not the setup time**
- Resolve within 1 ns: miss a window of $2.1 \cdot 10^{-16}$ s, light travels 62 nm
- t_r sits in the **exponent**: a few 100 ps change the MTBF by orders of magnitude

Synchronizers



The Two-Flip-Flop Synchronizer



- $t_r = T - t_{cq0} - t_{su}$, a third flip-flop multiplies the MTBF by $\exp(t_r/\tau)$
- Register in the source domain, **nothing between FF1 and FF2**, synchronize **once**
- Keep a level about **1.5 periods** of the destination, FF1 and FF2 close together

Pulses Across Domains

```
// domain A: every pulse toggles a level
if (pulse_a_i) tgl_a_q <= ~tgl_a_q;
// the level crosses through sync_2ff
sync_2ff u_sync (.clk_i(clk_b_i), ...,
  .d_i(tgl_a_q), .q_o(tgl_b));
// domain B: every change is a pulse
tgl_b_q <= tgl_b;  // a register
assign pulse_b_o = tgl_b ^ tgl_b_q;
```

- 1000 pulses from 10 ns into 14.3 ns, 50 ns apart
- **1000** arrive through `pulse_sync`
- **699** through a plain synchronizer

Exercise (Section 8.4): how long until a synchronizer fails, in the typical and the slow corner?

Crossing With Several Bits



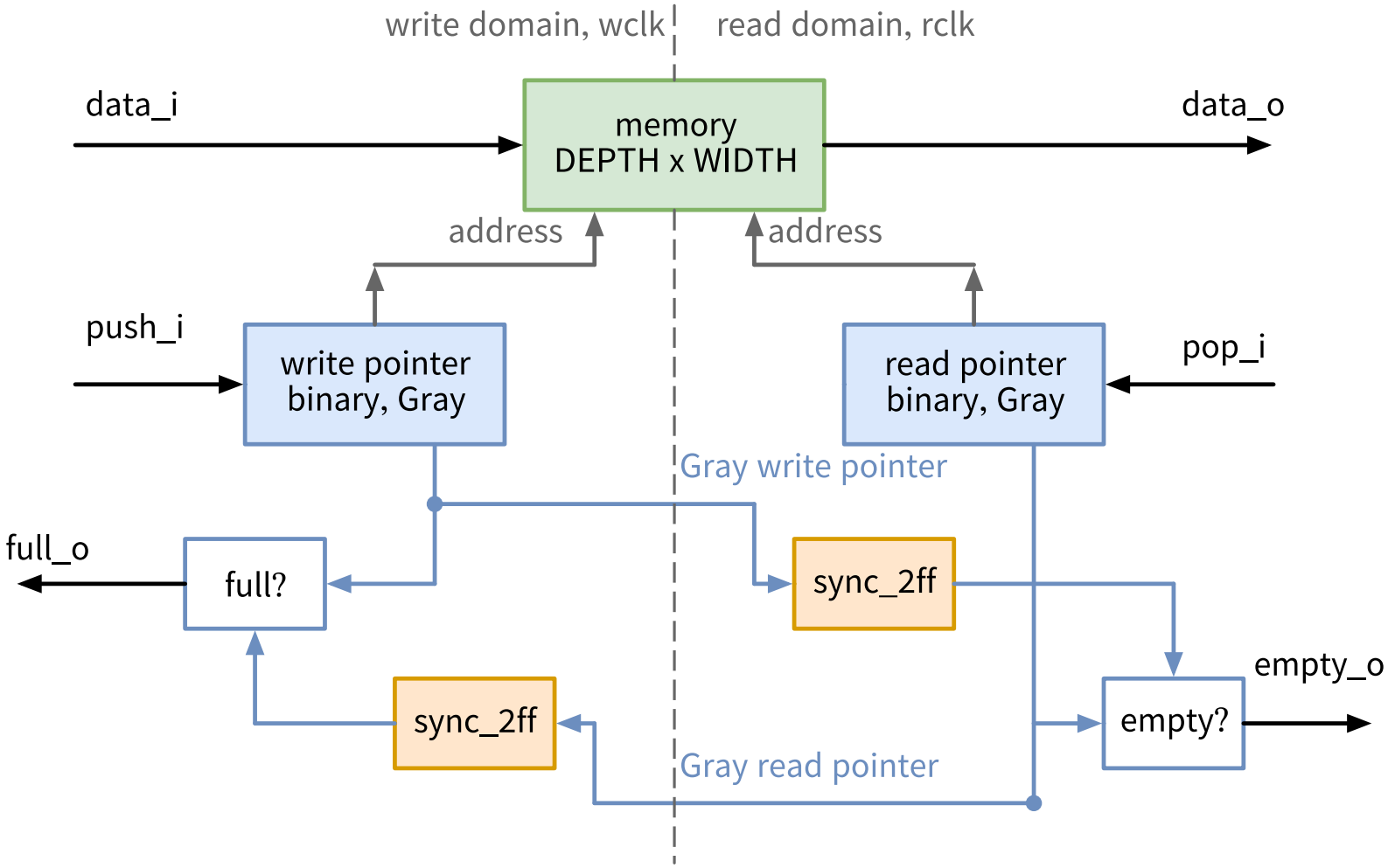
A Bus Through Synchronizers?

- Each bit resolves **on its own**: a mix of old and new
  - `0111` to `1000` can arrive as anything from 0 to 15
- **Gray code**: one bit per step, old or new value
  - $g = b \oplus (b \gg 1)$
- **Handshake**: data stable, request and acknowledge
- **Asynchronous FIFO** for data streams
- RTL simulation does not model metastability: `sync_2ff_meta.sv` **injects** it

n	binary	Gray
0	000	000
1	001	001
2	010	011
3	011	010
4	100	110
5	101	111
6	110	101
7	111	100

Exercise (Section 8.5): count across clock domains, in binary and in Gray code.

The Asynchronous FIFO



Full, Empty, and the Memory

- Pointers one bit wider than the address, as in Chapter 4
- **Full**: Gray codes of n and n + DEPTH differ in the **two upper bits**
  - 0 = `000` and 4 = `110` for DEPTH = 4
- Synchronized pointers lag: flags **pessimistic**, never low when full or empty
- Memory without synchronizer: read only after the pointer announced it
- With injection, 10 ns and 14.3 ns: **9397 pushed, 9395 popped, 0 errors**

Constraints for Clock Domain Crossings



SDC for Two Clocks

- Without further constraints, STA treats the clocks as **synchronous: common period 1430** ns
  - edges at 100.0 and 100.1 ns: memory to `data_o` fails with −1.414 ns, meaningless

```
set_clock_groups -asynchronous -group wclk -group rclk
# or, stricter: limit every crossing path, no hold check
set_max_delay 10.0 -ignore_clock_latency \
  -from [get_clocks wclk] -to [get_clocks rclk]
set_false_path -hold -from [get_clocks wclk] -to [get_clocks rclk]
# and the same from rclk to wclk
```

- Clock groups hide **every** crossing, also one **without** synchronizer
- Structure: CDC tools, or list all crossing endpoints: synchronizer or protocol

Resets



Releasing a Reset

- Asynchronous reset: **asserting** at any moment is safe
- **Releasing** is a crossing: recovery and removal (Chapter 6)
  - some flip-flops leave the reset one edge later: a state the RTL never reaches
- **Reset synchronizer**: assert asynchronously, release **synchronously**
  - two flip-flops with asynchronous reset, the first loads 1
- One per clock domain, and a **glitch-free** reset input

Exercise (Section 8.8): write a reset synchronizer, and time the barrel shifter with it.

Summary

- Crossings between asynchronous clocks: STA **cannot** check them
- Metastability resolves with τ: **MTBF** exponential in t_r
- **Two flip-flops** per level, pulses as toggled levels
- Buses: **Gray code**, handshake, **asynchronous FIFO** with Gray pointers
- Simulate with **metastability injection**
- SDC: clock groups or max delay, structure checked otherwise
- **Reset synchronizer**: assert asynchronously, release synchronously

Exercises

1. How Long Until It Fails? (8.4)
2. Count Across Clock Domains (8.5)
3. Release the Reset (8.8)

All code in `content/cdc/code` of the course repository, every number reproduced by `scripts/check_cdc.sh`



Chapter 8



JOHANNES KEPLER
UNIVERSITY LINZ